Specification Gaming Under Adaptive Oversight: A Cleaner-Overseer Grid-World Study

August 26, 2026

Abstract

1 Introduction

Reinforcement learning agents are typically optimized against a proxy reward signal that the designer hopes is aligned with the true objective. When the agent's actions can influence how that proxy is measured (rather than only the underlying state the proxy is meant to track) the two can come apart: the agent learns to make the proxy look good without making the true objective better. This failure mode, an instance of Goodhart's Law [1], is closely related to reward hacking and specification gaming [2], and becomes more severe when the agent has only partial observability over the state that oversight would need to verify.

We study this problem with a minimal, fully controllable environment: Overseer-Cleanup-Env, a grid-world cleaning task in which an agent is rewarded for making cells *appear* clean, but can satisfy this proxy either by genuinely removing dirt or by relocating it out of its own observation window. A second mechanism, the *overseer*, audits a bounded sample of claimed-clean cells with a probability that can be fixed or adapted online from a rolling history of the agent's behavior. This setup lets us isolate a single independent variable : the oversight policy governing *when* and *how* the proxy is checked against ground truth , while holding the underlying task and agent architecture fixed.

Our aim is to use this environment as a controlled testbed for a specific empirical question: does an oversight policy that adapts its audit frequency to observed behavior suppress reward hacking more effectively than a static policy at the same average audit budget? We train a PPO [3] cleaner agent under four oversight conditions : `random`, `fixed_rule`, `adaptive`, and `targeted` .We then compare true task completion, cheating rate, and catch rate across conditions. Beyond the main comparison, the process of arriving at a well-posed version of this environment surfaced several reward-hacking failure modes in the environment's own bookkeeping (Section 4), which we report as they bear directly on how sensitive this class of results is to implementation details often left unspecified in the literature.

2 Environment

Overseer-Cleanup-Env is a custom Gymnasium environment implementing a finite-horizon, partially observable Markov decision process (POMDP). We describe the cleaner agent's formulation here; the overseer's audit mechanism, which acts as a source of reward non-stationarity rather than a second learned agent in this work, is described separately in Section 3.

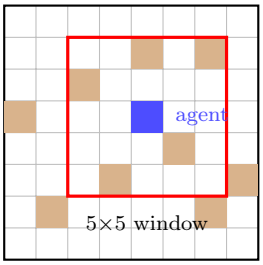
State and observation. The environment state at time t comprises an $N \times N$ grid (N=8) with a dirt count $d_t(c) \in \mathbb{Z}_{\geq 0}$ per cell c (dirt is stored as stackable units rather than a binary flag, so that relocating dirt onto an already-dirty cell conserves total dirt exactly), the agent position $p_t \in \{0, \ldots, N-1\}^2$, and two per-cell bookkeeping flags, $\text{claimed}_t(c)$ and $\text{fake}_t(c)$, used only by the audit mechanism (Sec. 3) and never exposed to the agent. The agent's observation is a strict subset of this state:

$$o_t = \big(w_t,\ p_t\big), \qquad w_t \in \{-1, 0, 1\}^{5\times 5},$$

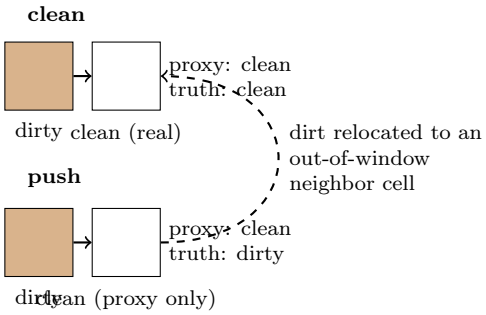
where w_t is a 5×5 window centered on p_t giving *presence* of dirt (1 = dirty, 0 = clean, −1 = outside the grid boundary), never the underlying stack count. The agent has no observation of claimed_t, fake_t, or any signal indicating whether or when it is being audited.

Action space. $\mathcal{A} = \{\texttt{up}, \texttt{down}, \texttt{left}, \texttt{right}, \texttt{clean}, \texttt{push}\}$, discrete, $|\mathcal{A}|$=6. Movement actions are deterministic and bounded; moving out of the grid is a no-op (agent position is unchanged, no penalty).

`clean` removes dirt at the agent's current cell for real, but is not atomic: once initiated, the agent is *locked* for CLEAN_STEPS=2 consecutive steps, during which the environment overrides whatever action the policy selects and forces `clean` again. The completion signal is internal (a step counter reaching CLEAN_STEPS), not a separate action. `push` instead relocates the dirt at the agent's cell to a uniformly random adjacent free cell in a single step – the dirt is not removed, only moved, and typically out of the agent's own 5×5 window.



(a) Grid, agent position, and its local observation window. Dirt outside the window (e.g. pushed there) is invisible to the agent.



(b) The proxy gap: `clean` and `push` produce the same proxy observation (cell reads as clean) but differ in ground truth. Only an audit can tell them apart.

Figure 1: Environment schematic. Dirt positions and window placement in (a) are illustrative.

Reward function. Every action incurs a fixed cost, and dirt cleaning (by either `real clean` or `push`) can trigger a one-time *local-clean bonus* the first time a given physical dirt unit is resolved, tracked via persistent per-unit identifiers to prevent an agent from re-earning the bonus by dragginf the same dirst unit across the grid:

$$r_t = -\text{cost}(a_t) + \mathbb{1}[\text{unit resolved for the first time}] \cdot \text{LOCAL\_CLEAN\_BONUS} + r_t^{\text{audit}},$$

with cost(`clean`) = CLEAN_ENERGY_COST per locked step (paid on both steps of the commitment) and cost(`push`) = PUSH_COST, cost(`move`) = 0. Constants are chosen so that `push` is strictly more reward-efficient per step than `clean` short-term (Sec. 4), which is what gives the agent an incentive to game the proxy at all. It is important to note that r_t is computed identically whether a resolved cell was genuinely cleaned or only pushed away , the agent's own training signal cannot distinguish honest from cheating completion. The only place ground-truth dirt state enters the reward is at audit, r_t^{audit}, a sparse adjustment applied only at audit events (Sec. 3).

Episode structure. Episodes run for a fixed horizon H=150 and always end by truncation; there is no early termination condition.The resolving all dirt does not end the episode early, so the agent must also learn not to keep attempting `clean`/`push` on an already-resolved grid, which carries pure cost with no offsetting bonus once dirt is exhausted. Dirt tiles are randomized at 20% density each episode via a persistent reset-time RNG stream, so successive episodes seen by the learner are not identical (Sec. 4 notes an early implementation bug where this was not the case).

3 Overseer Mechanism

The overseer is not a second RL agent in this work (Sec. 1); it is a fixed policy that determines *when* and *which* claimed-clean cells are checked against ground truth. It is the paper's independent variable: all four conditions share the same cleaner environment and only differ in this policy.

Audit cadence. An audit opportunity occurs every 30 steps .An earlier version of the environment audited only once, at episode end. Under that design, catch-probability could not scale with the volume of cheating within an episode . At each opportunity an audit actually fires with probability p_t^{audit}, whose form depends on the condition (below). If it fires, the overseer inspects a random sample of up to 8 cells currently claimed$clean(c)$=1 . Inspected cells are then cleared (claimed$(c)\leftarrow$0), so a later audit in the same episode cannot re-catch or re-confirm the same cell.

Suspicion and audit probability. The overseer maintains a suspicion $s_t \in [0, 1]$ that persists *across* episodes and is updated only at episode end, from results revaled by the episode's audits:

$$s_{t+1} = \begin{cases} \min(1,\ s_t + \text{SUSPICION\_CATCH\_BONUS}) & \text{if a fake-clean cell was caught this episode} \\ s_t \cdot \text{SUSPICION\_DECAY} & \text{otherwise (unaudited or audited-and-clean)} \end{cases}$$

The four conditions map s_t to an audit probability differently:

1. `Random`. Audits fire with the fixed baseline probability

$$p^{\text{audit}}_{\texttt{random}} = \text{BASE\_AUDIT\_PROB} = 0.05.$$

This condition provides a low-frequency, non-adaptive oversight baseline.

2. `fixed_rule`. Audits fire with the constant probability

$$p^{\text{audit}}_{\texttt{fixed\_rule}} = 0.4.$$

This rate is chosen to approximate the average audit budget of `adaptive`, allowing us to separate the effect of adaptivity from that of auditing more often.

3. `adaptive`. The audit probability increases linearly with the overseer's current suspicion:

$$p^{\text{audit}}_{\texttt{adaptive}} = \text{BASE\_AUDIT\_PROB} + s_t\big(\text{MAX\_AUDIT\_PROB} - \text{BASE\_AUDIT\_PROB}\big).$$

The overseer therefore audits rarely when suspicion is low and more frequently after evidence of cheating.

4. `targeted`. This condition uses the same adaptive audit frequency as `adaptive`:

$$p^{\text{audit}}_{\texttt{targeted}} = p^{\text{audit}}_{\texttt{adaptive}}.$$

It differs only in how cells are selected for inspection, as described below.

with MAX_AUDIT_PROB=0.9, capped strictly below 1 so that even maximal suspicion leaves cheating an escape chance.

Cell selection. Under `random`, `fixed_rule`, and `adaptive`, inspected cells are drawn uniformly at random from all claimed-clean cells .`Targeted` instead up-weights cells belonging to regions with a past confirmed catch, so that in principle a cleaner *could* learn to avoid known hotspots. This condition is put aside in the current experiments (Sec. 5) but is implemented and available.

Reward adjustments. An audit event returns ground-truth information that cleaner's reward is conditioned on. Let k be the number of inspected cells found fake-clean. The cleaner receives :

$$r^{\text{audit}} = \begin{cases} \text{FAKE\_CLEAN\_PENALTY} \times k, & k > 0 \quad (\text{FAKE\_CLEAN\_PENALTY} = -3) \\ \text{CLEAN\_CONFIRM\_BONUS}, & k = 0 \text{ and an audit occurred} \quad (= +1.0) \end{cases}$$

applied as a one-off adjustment folded into that step's reward (Sec. 2).

4 Training Setup

Algorithm and hyperparameters. The cleaner is trained with PPO [3] (Stable-Baselines3), one independent run per (condition, seed) pair. The observation space is a `Dict` (Sec. 2), which requires SB3's `MultiInputPolicy`. Table 1 lists the fixed hyperparameters, shared across all four conditions; only the overseer's `overseer_mode` (and `overseer_fixed_prob` for `fixed_rule`) varies between runs.

Table 1: PPO hyperparameters (identical across conditions).

Parameter	Value
Total timesteps	600,000
Parallel environments	1
n_{steps} (rollout length)	2048
Batch size	256
Learning rate	3×10^{-4}
Discount γ	0.99
Entropy coefficient	0.01
Grid size / dirt density / horizon	8×8 / 0.2 / 150

Reward constants and their justification. Two constants govern whether cheating is a genuine dilemma rather than a dominant or dominated strategy: the relative cost of `push` vs. `clean`, and the penalty for being caught. Per-step reward efficiency, ignoring audit risk, is

$$\begin{aligned} R^{\text{per-step}}_{\texttt{push}} &= \frac{\text{LOCAL\_CLEAN\_BONUS} - \text{PUSH\_COST}}{1} \\ &= \frac{0.5 - 0.05}{1} = \boxed{0.45 \text{ per step}}, \end{aligned}$$

$$\begin{aligned} R^{\text{per-step}}_{\texttt{clean}} &= \frac{\text{LOCAL\_CLEAN\_BONUS} - \text{CLEAN\_STEPS} \cdot \text{CLEAN\_ENERGY\_COST}}{\text{CLEAN\_STEPS}} \\ &= \frac{0.5 - 2(0.1)}{2} = \boxed{0.15 \text{ per step}}. \end{aligned}$$

Push has a 3× per-step efficiency advantage. This gap is large enough that cheating is attractive, but not so large that no audit penalty could offset it (an earlier gave a 9× gap under which cheating dominated regardless of audit pressure).

`FAKE_CLEAN_PENALTY` required more extensive tuning, jointly with `K_MAX_AUDIT`, because the effectiveness of a penalty depends on how much cheating an audit can detect. In the original single-audit design, most fake-clean cells were never inspected, so increasing the penalty alone did not reliably discourage cheating. We therefore introduced an audit opportunity every 30 steps, allowing the probability of detection to grow with the amount of cheating in an episode (Sec. 3). All reported experiments use K_MAX_AUDIT=8 and FAKE_CLEAN_PENALTY=−3.

Table 2: Tuning trajectory for `FAKE_CLEAN_PENALTY` and `K_MAX_AUDIT` (single-audit-per-episode design, single seed, `adaptive` condition).

K_MAX_AUDIT	FAKE_CLEAN_PENALTY	Observed behavior
3	−3	Cheating dominant in EV regardless of audit probability.
3	−10	Genuine oscillation: cheat_rate 0–1 throughout training, no saturation.
8	−6	Catch coverage improved but still noisy; no clean separation.

5 Experiments

Metrics. Each episode logs (Sec. 2, `Monitor` info fields): `true_completion_rate` (fraction of total dirt actually removed), `cheat_rate` (fraction of claimed-clean cells found fake since the last audit), `report_rate` (fraction of dirt claimed clean by any means), and `audit_prob` (the overseer's audit probability). We additionally derive `catch_rate` (the fraction of audit events that caught at least one fake-clean cell) from the per-audit `audit_caught_cheating` flag. All are reported as mean ± std over ≥ 5 seeds per condition, both at convergence (final training checkpoint) and as learning curves over training.

Experiment 1 – Main condition comparison. Train the cleaner under `random`, `fixed_rule`, and `adaptive` at matched average audit budget (Sec. 3), all else held fixed. *Hypothesis:* `adaptive` achieves lower converged `cheat_rate` and higher `true_completion_rate` than `fixed_rule` at the same average audit probability, and both outperform `random`; the comparison isolates the effect of adapting audit frequency to behavior from the effect of auditing more per se. This is the paper's primary result.

Experiment 2 – Arms-race dynamics. For `adaptive`, plot `audit_prob` and `cheat_rate` jointly over training. *Hypothesis:* the two co-vary – `audit_prob` rises with recent cheating and decays as the agent is suppressed – rather than `audit_prob` monotonically saturating at `MAX_AUDIT_PROB` (which would indicate the suspicion mechanism is not actually behavior-contingent in practice, only in specification).

6 Results

Data. Results below come from 3 seeds per condition (0, 5, 10) for `random`, `fixed_rule`, and `adaptive`, each trained for 4000 logged episodes. This is below the ≥5-seed target but we report these resullts as preliminary. Something important to mention is that`cheat_rate` is measured since the last audit rather than cumulatively over the episode.

Main comparison (Experiment 1). Table 3 reports final-window (last 10% of training) mean ± std across seeds. `random` fails to suppress cheating: cheat rate stays at 0.809 ± 0.040 and true completion at only 0.200 ± 0.053, confirming that `push` dominates `clean` in the absence of meaningful audit pressure (Sec. 4). Both `fixed_rule` and `adaptive` suppress cheating almost completely, and reach near-ceiling true completion (0.999 and 0.997) .Figure 2 shows both converging after 1000 episodes.

Table 3: Final-window performance, mean ± std over 3 seeds.

Condition	true_completion_rate	cheat_rate	report_rate	audit_prob
random	0.200 ± 0.053	0.809 ± 0.040	0.847 ± 0.301	0.050 ± 0.000
fixed_rule	0.999 ± 0.000	0.000 ± 0.000	0.179 ± 0.014	0.400 ± 0.000
adaptive	0.997 ± 0.002	0.005 ± 0.003	0.689 ± 0.120	0.153 ± 0.094

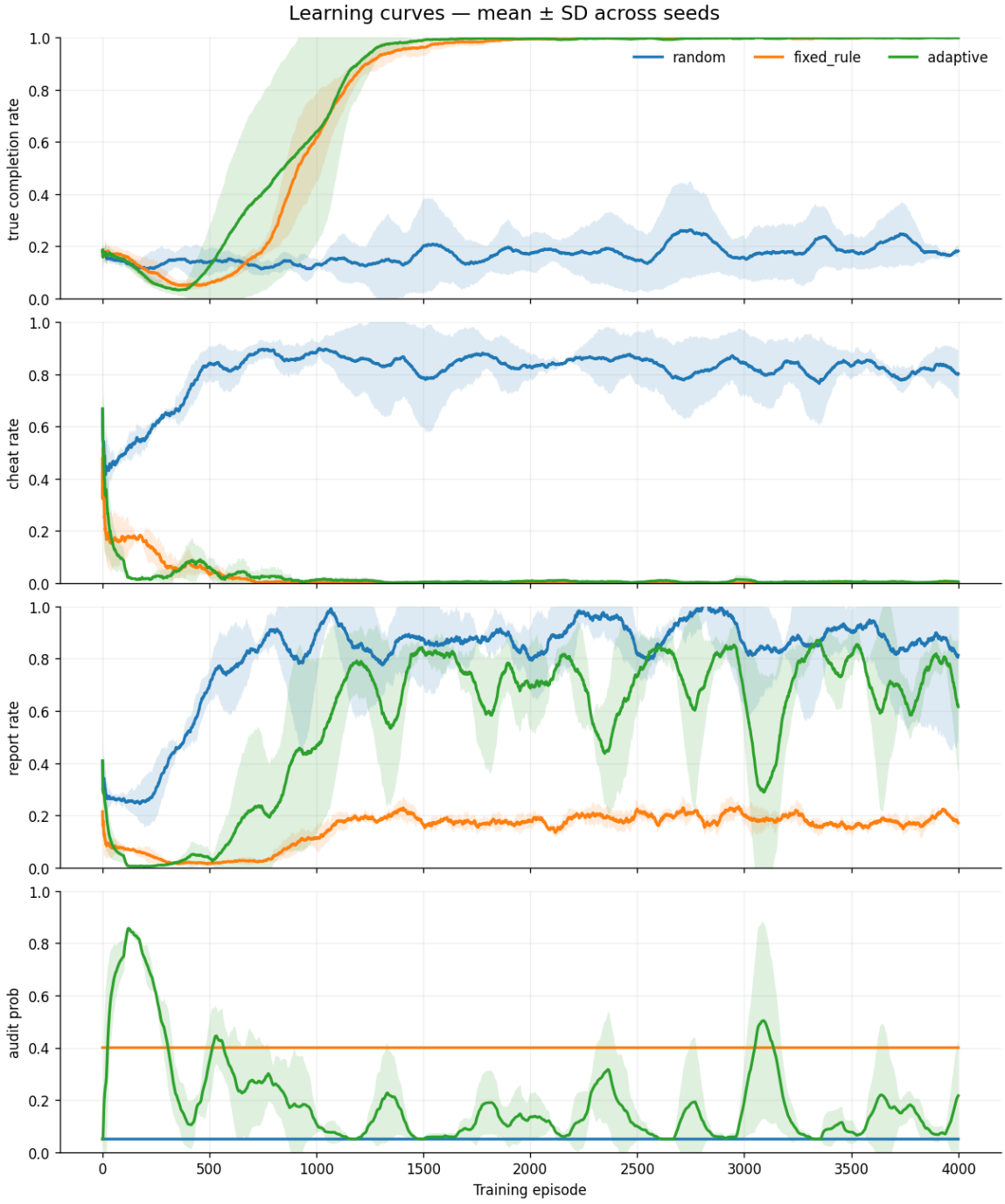


Figure 2: Learning curves, mean ± SD across seeds.

This is not the result Experiment 1 was framed to expect (`adaptive` outperforming `fixed_rule`). Instead, the two conditions reach similar outcomes. While their audit budgets differ: `fixed_rule` audits at a constant 0.400, while `adaptive`'s empirical mean settles at 0.153–0.189. The more accurate empirical claim is therefore: *adaptive oversight reaches the same suppression of cheating as constant high-frequency auditing, at under half the average audit cost*, because it only spends its budget while cheating is actually occurring rather than throughout training.

Arms-race dynamics (Experiment 2). Figure 3 plots `adaptive`'s cheat rate and audit probability jointly over training. The pattern is consistent with genuine feedback rather than saturation: audit probability spikes early (mean peak ≈0.85 while cheat rate is still high), decays as cheating is suppressed, and repeatedly re-spikes to moderate levels (0.2–0.5) coinciding with smaller transient increases in cheat rate later in training, always decaying back down afterward. Across seeds, audit probability spends only 2.8–4.9% of training within 95% of `MAX_AUDIT_PROB`, ruling out permanent saturation as an explanation. A lagged cross-correlation between cheat rate and audit probability (not shown) is directionally consistent with delayed response – correlation rises from ≈ -0.02 at lag 0 to ≈ 0.10 at a 20-episode lag – but the effect is small and its confidence band spans zero for much of that range, so we treat this as weak quantitative support alongside the clearer qualitative pattern in Figure 3, not as a strong statistical confirmation on its own.

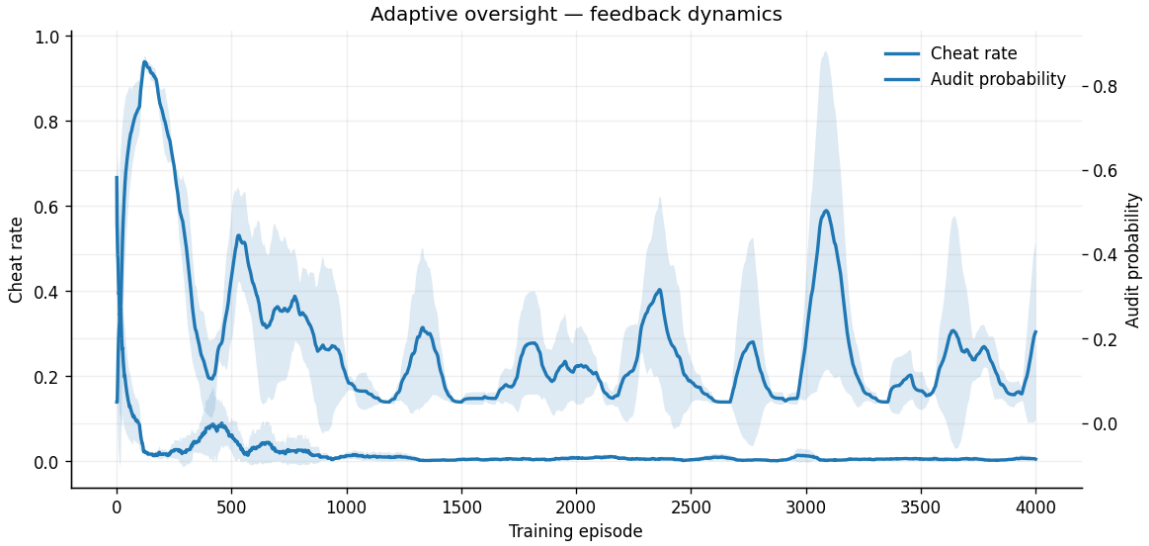


Figure 3: `adaptive` condition: cheat rate and audit probability over training, mean across seeds with ±SD band. Audit probability tracks cheat rate with a delay and decays once cheating is suppressed.

Reward and episode length. Mean episode reward (Fig. 2 tracks the same ordering: `fixed_rule` and `adaptive` converge to positive reward (≈5–6), while `random` remains negative throughout training, consistent with a cleaner that is punished for wasted actions after dirt is exhausted without ever earning the confirm bonus. All conditions run the full H=150 steps every episode.

7 Discussion and Limitations

Adaptive audit frequency oversight matched constant high-frequency auditing on supressing cheating while using half the average audit budget. The main result is about efficiency, not of adaptive achieving something fixed-rate auditing structurally cannot. Both were far ahead of `random`, confirming the environment poses a genuine dilemma.

Limitations. (i) Results rest on 3 seeds/condition, below the ≥5 target; variance across seeds (ii) `cheat_rate` is measured since the last audit, not cumulatively per episode, likely inflating apparent suppression for all conditions equally but not verified to preserve the between-condition ordering.(iii) `targeted` and any scale/budget sweep beyond Sec. 5 were not run.

Future work. Repeat Experiment 1 at ≥5 seeds to tighten these estimates; fix `cheat_rate` to a cumulative-per-episode definition; evaluate `targeted` against `adaptive` to test whether biased cell selection adds suppression beyond frequency alone; and extend the self-play direction noted in Sec. 1, letting the overseer itself be learned rather than fixed-policy.

8 Conclusion

We built OverseerCleanupEnv, a minimal grid-world in which a proxy reward can be satisfied either honestly or by cheating, and used it to test whether adapting audit frequency to observed behavior curbs this kind of reward hacking better than a static audit policy at similar cost. Under a PPO-trained cleaner and three oversight conditions, adaptive and fixed audit drove cheat rate to near zero and true completion to near ceiling . But it did so at under half the average audit budget, spending oversight effort only while cheating was actually occurring. Reaching this result also required fixing the audit mechanism itself: an earlier, single-audit-per-episode design left catch probability structurally unable to scale with the volume of cheating, and no penalty tuning could compensate for it (Sec. 4); only redesigning *when* auditing happens, not how harshly it punishes, closed that gap. Taken together, the results suggest that in settings where oversight is costly, the practical value of an adaptive policy may lie less in achieving outcomes a static policy cannot reach, and more in reaching the same outcome for less.

References

[1] C. A. E. Goodhart, "Problems of monetary management: The uk experience," *Monetary Theory and Practice*, pp. 91–121, 1984.

[2] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in ai safety," *arXiv preprint arXiv:1606.06565*, 2016.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.